

# Comparison of speed and performance between Python and R implementations of the AddiVortes algorithm

Lewis Maltby

June 30, 2026

## 1 Introduction

The purpose of this article is to compare the speed and performance of the AddiVortes algorithm between the current up-to-date Python and R implementations. We will benchmark the fitting time of a model, as well as the amount of time required to make simple predictions on a test set, to try to determine which of the two implementations is faster to use. We will also benchmark the in-sample and out-of-sample RMSE on that data to determine which of the two implementations gives predictions with lower errors.

The code used to generate the benchmarking data, as well as the generated data itself and all referenced files, are available to view at <https://github.com/snowy-the-leopard/AddiVortes>. This data was generated using the ADDIVORTES R package, version 0.6.5, available to view here <https://github.com/johnpaulgosling/AddiVortes>, and the ADDIVORTES Python package, version 0.6.5, available to view here <https://github.com/johnpaulgosling/py-addivortes>.

## 2 Benchmarking methodology

To ensure fairness, consistency, and accuracy with these tests, the following controls were implemented on these benchmarks:

1. The benchmarks were ran on a new GitHub Codespace, a clean installation of Python and R were used, with only the necessary packages downloaded to minimise background resource usage. The Codespace used is a 2-core 2.8Ghz system with 8GB of memory. Exact details of hardware used can be found at [AddiVortes/mySpecs.html](#).
2. The code was set up such that each implementation used only a single thread, to counteract the potentially different methods of multi threading between the two languages. This choice was made as we are working under the assumption that when we are working with HPC resources, the code will be optimised to allow multiple threads to be used at once. This choice allows us to compare algorithmic efficiency in each implementation more directly.
3. The same models and predictions were fitted repeatedly in order to capture the small variations in fit and prediction times between runs. This will also produce a set of slightly varying prediction results due to the random nature of the MCMC fitting algorithm, and allow us to better analyse how consistent our results are.
4. The benchmarks were ran sequentially rather than simultaneously to minimise the risk of shared resources skewing the results, where one script could take computational priority over another, leading to a larger discrepancy in results.
5. The timings were taken only between the computation required for fitting and generating predictions, with the pre-generated datasets already loaded in prior to this.

### 3 Datasets used in benchmarking

To allow for a wide scope of comparison between the two implementations, the following 4 different datasets were used in benchmarking:

#### 3.1 Boston dataset

The Boston Housing Dataset, derived from information from the U.S. Census Service. This consists of 13 numerical covariates, and we model it using the following fixed parameters, with the rest of the parameters taking their default values: `m = 50`

#### 3.2 Synthetic dataset

We have constructed a synthetic dataset using noisy functions of independently and identically distributed  $U[0, 1]$  observations. This consists of 5 numerical covariates, and we model it by taking all the parameters as their default values.

#### 3.3 Spherical dataset

We have constructed random longitudes and latitudes to represent positions on a sphere, and a noisy response function. This consists of 2 numerical covariates (the coordinates), and we model it using the following fixed parameters, with the rest of the parameters taking their default values: `m = 50`, `totalMCMCIter = 500`, `mcmcBurnIn = 100`, `metric = "S"`. Note that this uses great circle distance, the shortest distance between two points over the surface of the sphere.

#### 3.4 Categorical dataset

We have constructed this dataset using random numerical and categorical covariates, as well as a noisy response function. This consists of 2 numerical and 2 categorical covariates, and we model it using the following fixed parameters, with the rest of the parameters taking their default values: `m = 50`, `totalMCMCIter = 500`, `mcmcBurnIn = 100`, `catScaling = 1`

### 4 Benchmark results